UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA
FACULTAD DE CIENCIAS QUÍMICAS E INGENIERÍA
INGENIERÍA EN SOFTWARE Y TECNOLOGÍAS EMERGENTES

Gestión de la Innovación - 40027

Práctica de taller 5

Rosales Quiñones Fernando - 129940

7 de abril de 2026

14 de abril de 2026

INTRODUCCIÓN 4
OBJETIVOS 6
Objetivo General 6
Objetivos Específicos 6
I. Marco Teórico 7
1.1 Las Capas de la Ingeniería de Software: Una Base Histórica 7
1.2 De la Mecanización a la Era de la Información 8
II. DESARROLLO: ANÁLISIS DEL IMPACTO TECNOLÓGICO 9
2.1 Análisis de la Línea del Tiempo: Hitos Clave 9
2.2 Síntesis de Impacto: El Transistor y ARPANET como Pilares de la Ingeniería de Software Moderna 12
2.3 Línea del tiempo 14
2.4 Análisis Contrafáctico: ¿Qué habría ocurrido sin el Transistor y ARPANET? 15
Escenario A: Sin el Transistor 15
Escenario B: Sin ARPANET 16
III. ACTIVIDADES COMPLEMENTARIAS 17
3.1 Ensayo: El Alan Turing del Futuro 17
3.2 Reflexión: Ada Lovelace y su Visión de la Inteligencia Artificial 18
3.3 Algoritmo de Fibonacci Representado en Tarjetas Perforadas 21
CONCLUSIONES 23
REFERENCIAS 26

Resumen

La Ingeniería de Software es la culminación de milenios de abstracción y automatización, consolidándose como la disciplina esencial para gestionar la complejidad sistémica que define nuestra era. El paso de la mecanización industrial a la era de la información fue impulsado por hitos como el transistor, que masificó el hardware, y ARPANET, que globalizó el software, creando la necesidad de métodos rigurosos para garantizar la calidad y la distribución de los datos. En este contexto, el ingeniero actúa como un puente entre la teoría y la práctica, donde entender los fundamentos históricos desde la visión de Ada Lovelace hasta los modelos de IA actuales resulta clave para orquestar con ética y juicio las tecnologías que sostienen la infraestructura del futuro.

INTRODUCCIÓN

La Ingeniería de Software no emergió de manera abrupta ni como resultado de un único descubrimiento aislado. Por el contrario, representa la culminación de miles de años de esfuerzo humano orientado hacia la abstracción, la automatización y la gestión sistemática de la complejidad. Para comprender en su totalidad el alcance y los fundamentos de esta disciplina, resulta imprescindible trazar su genealogía intelectual a través de las distintas eras tecnológicas que han configurado la civilización humana.

Desde las primeras herramientas de piedra talladas por el Homo habilis hace aproximadamente 2.5 millones de años que constituyen la primera instancia documentada de un proceso orientado a la solución de un problema mediante pasos definidos hasta los modernos sistemas de inteligencia artificial generativa, la humanidad ha construido un continuo de innovación en el que cada avance sienta las bases del siguiente. En palabras de Pressman y Maxim (2020), "la ingeniería de software abarca un proceso, un conjunto de métodos y un arreglo de herramientas que permite a los profesionales construir software de alta calidad" (p. 3), definición que solo cobra pleno sentido cuando se contextualiza históricamente.

El estudio del contexto socio-técnico en el que surgieron los principales hitos tecnológicos permite identificar los factores económicos, científicos y culturales que catalizaron cada transformación. Ceruzzi (2003) sostiene que la historia de la computación no puede separarse del contexto social e industrial en el que se desarrolló, pues las necesidades concretas de cada época desde el procesamiento del censo de 1890 hasta los cálculos balísticos de la Segunda Guerra Mundial dictaron la dirección de la innovación tecnológica.

La presente práctica analiza cinco grandes eras tecnológicas: la era prehistórica (2.5 M a.C. – 500 d.C.), que abarca desde las primeras herramientas hasta el sistema numérico posicional; la era medieval (500 – 1700), que consolida el álgebra y los primeros dispositivos mecánicos de cálculo; la era industrial (1700 – 1900), que introduce la mecanización y los primeros precursores directos de la

computadora; la era de los fundamentos (1900 – 1950), que establece los cimientos matemáticos y electrónicos de la computación moderna; y la era de la información (1950 – presente), que comprende desde las primeras computadoras comerciales hasta la inteligencia artificial generativa actual.

A través del análisis de la línea del tiempo interactiva elaborada para esta práctica, se evidencia que la Ingeniería de Software es, en esencia, la respuesta disciplinaria formal a una crisis de complejidad creciente: la necesidad de construir sistemas de software confiables, mantenibles y eficientes en un contexto de expansión tecnológica sin precedentes. Como señala Sommerville (2011), la ingeniería de software surge precisamente porque el software se ha convertido en un componente crítico de prácticamente todos los sistemas técnicos del mundo moderno.

OBJETIVOS

Objetivo General

Analizar la evolución histórica de la tecnología desde sus manifestaciones más primitivas hasta la inteligencia artificial contemporánea, con el propósito de comprender el contexto socio-técnico, económico y científico que propició el surgimiento y consolidación de la Ingeniería de Software como disciplina formal de la ingeniería.

Objetivos Específicos

- Identificar y describir los hitos tecnológicos fundamentales a lo largo de cinco eras históricas desde las herramientas de piedra del Homo habilis hasta los sistemas de inteligencia artificial generativa estableciendo las relaciones de causalidad y dependencia entre cada avance y sus sucesores inmediatos.
- Evaluar las condiciones socioeconómicas, culturales y científicas que propiciaron el surgimiento de cada innovación tecnológica significativa, reconociendo el papel que desempeñaron las necesidades militares, comerciales y académicas como motores del desarrollo en la historia de la computación, con especial énfasis en el período 1900–1960.
- Determinar los grupos sociales, instituciones y actores que se beneficiaron primariamente de cada avance tecnológico, así como las transformaciones estructurales que dichos avances generaron en la organización del trabajo, la distribución del conocimiento y la estructura de la industria del software desde la era industrial hasta la actualidad.

I. Marco Teórico

1.1 Las Capas de la Ingeniería de Software: Una Base Histórica

Pressman y Maxim (2020) describen la Ingeniería de Software como una tecnología con capas superpuestas: en la base se encuentra el compromiso con la calidad; sobre ella se apoya el proceso que define el marco de trabajo; encima se encuentran los métodos que proveen el "cómo hacer"; y en la capa superior residen las herramientas que proporcionan soporte automatizado o semiautomatizado al proceso y los métodos. Esta arquitectura conceptual refleja con exactitud el desarrollo histórico de la disciplina.

La capa más profunda del compromiso con la calidad tiene sus raíces en la crisis del software de la década de 1960. La Conferencia NATO de 1968, celebrada en Garmisch, Alemania, fue el evento fundacional de la disciplina precisamente porque los sistemas de software de la época, paradigmáticamente el OS/360 de IBM, eran costosos, poco confiables y difícilmente mantenibles. Brooks (1995) documentó magistralmente esta problemática en "The Mythical Man-Month", argumentando que la complejidad esencial del software aquella inherente a la naturaleza del problema que resuelve no puede eliminarse, solo gestionarse mediante técnicas de ingeniería rigurosas.

La capa del proceso tiene sus antecedentes históricos en las metodologías industriales del siglo XIX: la organización científica del trabajo de Taylor, la producción en serie de Ford y los principios de gestión de proyectos que surgieron de las grandes obras de ingeniería civil. Cuando los proyectos de software comenzaron a alcanzar escalas comparables a las de las grandes obras de infraestructura, resultó inevitable aplicar principios similares de planificación, control y aseguramiento de la calidad.

Las capas de métodos y herramientas reflejan la evolución del hardware. Cada generación de hardware tubos de vacío, transistores, circuitos integrados, microprocesadores impuso nuevas restricciones y abrió nuevas posibilidades al desarrollo de software. Como señala Pressman y Maxim (2020), la evolución del hardware ha sido el principal catalizador de la demanda de métodos y

herramientas de ingeniería de software más sofisticados, pues el incremento exponencial en la capacidad de cómputo permitió abordar problemas progresivamente más complejos.

1.2 De la Mecanización a la Era de la Información

La transición desde la manufactura industrial a la era de la información digital no fue un salto discontinuo, sino una cadena de transformaciones graduales en las que cada innovación preparó el terreno para la siguiente. Ceruzzi (2003) identifica tres grandes transiciones en esta trayectoria: la mecanización del cálculo (siglos XVII-XIX), la electrificación del cálculo (primera mitad del siglo XX) y la digitalización masiva de la información (segunda mitad del siglo XX hasta el presente).

La primera transición comenzó con las calculadoras mecánicas de Schickard (1623) y Pascal (1642), alcanzó su cúspide conceptual con la Máquina Analítica de Babbage (1837) y sentó las bases industriales con la Máquina Tabuladora de Hollerith (1890). Lo que caracteriza este período es la búsqueda de la automatización del trabajo intelectual repetitivo: el cálculo aritmético y el procesamiento de datos a gran escala. La fundación de IBM en 1924, heredera directa de la empresa de Hollerith, simboliza la consolidación de la industria del procesamiento de datos como actividad económica independiente.

La segunda transición fue acelerada decisivamente por la Segunda Guerra Mundial. Las necesidades de criptoanálisis (Colossus, 1943) y de cálculo balístico (ENIAC, 1945) justificaron inversiones gubernamentales masivas en el desarrollo de computadoras electrónicas. La formulación de la Arquitectura de Von Neumann (1945) y la invención del transistor (1947) en Bell Labs por Bardeen, Brattain y Shockley proporcionaron los fundamentos teórico y físico, respectivamente, de todas las computadoras digitales que seguirían.

La tercera transición, la digitalización masiva, se caracteriza por la convergencia de tres tendencias: la miniaturización continua de los componentes electrónicos (descrita por la Ley de Moore, 1965), la estandarización de los protocolos de comunicación (TCP/IP, 1974) y la democratización del acceso a la computación (Apple II, 1977; IBM PC, 1981). Estas tendencias culminaron en Internet y la World Wide Web, que transformaron el software de herramienta especializada a infraestructura global de la que depende la economía, la educación, la salud y la comunicación de miles de millones de personas.

II. DESARROLLO: ANÁLISIS DEL IMPACTO TECNOLÓGICO

2.1 Análisis de la Línea del Tiempo: Hitos Clave

El análisis sistemático de la línea del tiempo interactiva elaborada para esta práctica permite identificar un conjunto de innovaciones cuyo impacto sobre la Ingeniería de Software fue determinante. A continuación se presentan los hitos de mayor relevancia disciplinar, organizados en una tabla comparativa:

Hito	Autor / Institución	Nivel de Impacto	Relevancia para la Ingeniería de Software
Transistor (1947)	Bardeen, Brattain y Shockley Bell Labs	Revoluciona rio	Sustituyó el tubo de vacío. Habilitó la miniaturización masiva que haría posible los microprocesadores y toda la computación digital moderna.
Arquitectura Von Neumann (1945)	John von Neumann Princeton	Revoluciona rio	Definió el modelo de programa almacenado: CPU, memoria unificada

			y bus de datos. Toda computadora actual sigue este paradigma.
ARPANET (1969)	DARPA / Bolt, Beranek & Newman	Revolucionario	Primera red de conmutación de paquetes. Ancestro directo de Internet; instauró los principios de descentralización y resiliencia en las redes.
Intel 4004 (1971)	Ted Hoff, Federico Faggin Intel	Revolucionario	Primera CPU en un solo chip (2,300 transistores). Inauguró la era de la computación personal y los dispositivos embebidos.
TCP/IP (1974)	Vint Cerf y Bob Kahn DARPA	Revolucionario	Protocolo universal de intercomunicación de redes. Define el lenguaje que habla Internet; recibió el Premio Turing en 2004.
IBM PC (1981)	Don Estridge / IBM Florida	Revolucionario	Arquitectura abierta que estandarizó la plataforma PC y creó el ecosistema de software comercial masivo.

World Wide Web (1989)	Tim Berners-Lee CERN	Revolucionario	HTML, HTTP y URL democratizaron el acceso a la información. Berners-Lee donó el invento sin patentar. Generó una industria de billones de dólares.
Transformers (2017)	Vaswani et al. Google Brain	Revolucionario	Arquitectura de atención que sustituyó las RNN. Base de todos los modelos de lenguaje grandes (GPT, BERT, LLaMA).

La tabla evidencia que los hitos de mayor impacto se concentran en dos períodos críticos: el período 1945–1950, que establece los fundamentos matemáticos y electrónicos de la computación, y el período 1969–1981, que transforma la computación de actividad institucional a fenómeno masivo. Cabe destacar que la totalidad de los hitos clasificados con impacto revolucionario comparte una característica común: cada uno resolvió una limitación fundamental que impedía el siguiente nivel de escala o abstracción en el desarrollo de sistemas de software.

2.2 Síntesis de Impacto: El Transistor y ARPANET como Pilares de la Ingeniería de Software Moderna

Si hubiera que seleccionar dos innovaciones tecnológicas cuya ausencia habría impedido el surgimiento de la Ingeniería de Software tal como la conocemos, estas serían el transistor (1947) y ARPANET (1969). No se trata de una selección arbitraria, sino de una conclusión derivada del análisis causal de la cadena de dependencias tecnológicas que estructuran la historia de la computación.

El transistor, inventado por John Bardeen, Walter Brattain y William Shockley en los Laboratorios Bell en diciembre de 1947, resolvió el problema estructural más crítico de la computación electrónica de primera generación: la fragilidad, el tamaño y el consumo energético de los tubos de vacío. El ENIAC, la primera computadora electrónica de propósito general (1945), contenía 17,468 tubos de vacío, pesaba 30 toneladas y consumía 150 kilovatios de energía. Con esas características físicas, la computación era una actividad reservada a grandes instituciones gubernamentales o académicas con recursos extraordinarios. El transistor cambió radicalmente esta ecuación al ser más pequeño en varios órdenes de magnitud, más rápido, más fiable, y al consumir una fracción mínima de la energía de un tubo de vacío. La consecuencia directa fue el circuito integrado (1958), después el microprocesador (1971), y finalmente la computadora personal (1975-1981).

Esta progresión de miniaturización tuvo un efecto transformador sobre la naturaleza del software. Conforme los sistemas de hardware se volvían más accesibles y sus capacidades se expandían, los programas que los controlaban crecían en tamaño y complejidad a un ritmo que superaba la capacidad individual de cualquier programador. Como documenta Brooks (1995) en su análisis del proyecto OS/360 de IBM, la complejidad del software escala de manera no lineal: la adición de recursos humanos a un proyecto tardío no lo acelera, sino que lo retrasa, debido a los costos de coordinación y comunicación. Esta constatación fue precisamente la que motivó la búsqueda de metodologías

formales, lenguajes de programación de alto nivel, estructuras de datos y algoritmos eficientes, patrones de diseño y, en última instancia, la formalización de la Ingeniería de Software como disciplina en 1968.

ARPANET, por su parte, introdujo una dimensión radicalmente nueva: la distribución. Antes de ARPANET, el software era, por definición, un fenómeno local: un programa se ejecutaba en una única máquina y los datos residían en un único almacenamiento. La red de conmutación de paquetes concebida por Paul Baran y Donald Davies, e implementada por DARPA y Bolt, Beranek & Newman, estableció el principio de que la información puede fragmentarse en paquetes, enrutarse de manera independiente a través de múltiples nodos, y reensamblarse en el destino. Este principio, formalizado en el protocolo TCP/IP por Vint Cerf y Bob Kahn en 1974, transformó el software de artefacto local a infraestructura global.

Las implicaciones para la Ingeniería de Software fueron profundas y multidimensionales. En primer lugar, la distribución introdujo nuevas clases de problemas concurrencia, consistencia, tolerancia a fallos, latencia que requirieron nuevos paradigmas de diseño y nuevas metodologías de desarrollo. En segundo lugar, la conectividad en red facilitó la colaboración distribuida en el desarrollo de software, sentando las bases del movimiento de software libre (GNU, 1983), del código abierto (Linux, 1991; GitHub, 2008) y de las metodologías ágiles que dependen de equipos distribuidos globalmente. En tercer lugar, la escala global de Internet creó sistemas de una complejidad sin precedentes históricos: el buscador de Google procesa más de 8.5 mil millones de consultas diarias, Facebook gestiona datos de más de 3 mil millones de usuarios activos, y AWS sostiene una fracción significativa de la infraestructura digital mundial. Gestionar sistemas de esta magnitud requiere exactamente lo que la Ingeniería de Software proporciona: procesos rigurosos, métodos probados y herramientas especializadas.

En síntesis, el transistor hizo posible la computadora personal y, con ella, la necesidad masiva de software; ARPANET hizo posible Internet y, con ella, la necesidad de sistemas de software

distribuidos de escala global. La Ingeniería de Software es, en buena medida, la respuesta disciplinaria a los desafíos de complejidad que ambas innovaciones introdujeron.

2.3 Línea del tiempo

ERA I
Prehistoria

2.5M a.C. – 500 d.C.

~2.5 Millones a.C.

Primeras Herramientas de Piedra

Homo habilis — Olduvai, Tanzania

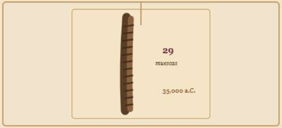
Las herramientas olduvayenses representan la primera tecnología humana. El acto de modificar una piedra para un propósito específico constituye el primer "programa": una secuencia de acciones...

IMPACTO: FUNDAMENTAL

~35,000 a.C.

Hueso de Lebombo — Primer Registro Numérico

Cultura San — Sudáfrica


29
muescas
35,000 a.C.

Un peroné de babuino con 29 muescas talladas, encontrado en las montañas Lebombo. Es considerado el primer artefacto matemático conocido — un sistema de conteo unario. La...

IMPACTO: FUNDAMENTAL

~3500 a.C.

Invención de la Escritura Cuneiforme

Civilización Sumeria — Mesopotamia

Los sumerios desarrollaron la escritura sobre tablillas de arcilla para registrar transacciones comerciales, inventarios fue la primera forma de almacenamien

IMPACTO: REVOLUCIONARIO

cuneiforme

y leyes. Esta
to de datos...

~3000 a.C.

El Ábaco

Mesopotamia / China

3000 a.C.

El ábaco es la primera herramienta de cálculo conocida. Permitía realizar operaciones aritméticas de suma, resta, multiplicación y división mediante la manipulación física de cuentas. Es el primer...

IMPACTO: ALTO

~300 a.C.

Elementos de Euclides

Euclides — Alejandría, Egipto

Elementos - Euclides 300 a.C.

Los 13 libros de los "Elementos" compilaron todo el conocimiento geométrico de la antigüedad de forma axiomática y deductiva. El algoritmo de Euclides para calcular el máximo común divisor...

IMPACTO: FUNDAMENTAL

~150 a.C.

Mecanismo de Anticitera

Ingenieros griegos — Grecia Antigua

Descubierto en un naufragio en 1901, este dispositivo con más de 30 engranajes de bronce predecía posiciones astronómicas, eclipses y ciclos olímpicos. Es considerada la primera computador...

IMPACTO: REVOLUCIONARIO

~250 d.C.

Sistema Numérico Posicional y el Cero

Matemáticos indios — India

1 0 9

1 0 1 0 = 10

Base 10 a Base 2 - 250 d.C.

La civilización india desarrolló el sistema decimal posicional con un símbolo para el cero (शून्य, śūnya). Sin el cero y la notación posicional, el sistema binario (base 2) que utilizan todas las...

`IMPACTO: REVOLUCIONARIO`

ERA II
Medieval

500 — 1700

820 d.C.

Álgebra y el Concepto de "Algoritmo"

Muhammad ibn Musa al-Juarismi — Bagdad

$$ax2 + bx + c = 0$$

Al-Juarismi

Origen de algoritmo - 820 d.C.

Al-Juarismi escribió "Al-Kitāb al-mukhtaṣar fī ḥisāb al-jabr wal-muqābala" (Compendio de cálculo por reintegración y comparación), que introdujo el álgebra como disciplina. Su nombre...

`IMPACTO: REVOLUCIONARIO`

1450

Invención de la Imprenta de Tipos Móviles

Johannes Gutenberg — Maguncia, Alemania

BIBLIA
Gutenberg
MCDL
1450

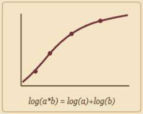
La imprenta de Gutenberg no fue solo una revolución en la comunicación: fue la primera tecnología de replicación masiva de información. El concepto de tipos móviles es un precursor directo...

`IMPACTO: REVOLUCIONARIO`

1614

Invención de los Logaritmos

John Napier – Escocia



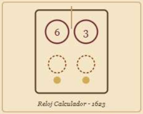
Napier publicó "Mirifici Logarithmorum Canonis Descriptio", introduciendo los logaritmos como herramienta para simplificar cálculos complejos. Convertir multiplicaciones en sumas aceleró...

`IMPACTO: ALTO`

1623

Primera Calculadora Mecánica

Wilhelm Schickard – Tubinga, Alemania


6
3
Reloj Calculador - 1623

Schickard diseñó el "Reloj Calculador", la primera calculadora mecánica conocida, capaz de sumar, restar, multiplicar y dividir números de hasta seis dígitos. Utilizaba engranajes y un mecanismo de...

`IMPACTO: ALTO`

1642

La Pascalina

Blaise Pascal – Francia

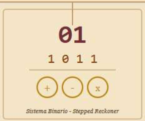
Con solo 19 años, Pascal inventó la Pascalina para ayudar a su padre con cálculos fiscales. Esta calculadora mecánica con ruedas dentadas podía sumar y restar. Fue la primera calculadora...

`IMPACTO: ALTO`

1673

Stepped Reckoner y el Sistema Binario

Gottfried Wilhelm Leibniz – Alemania


01
1 0 1 1
+
-
x
Sistema Binario - Stepped Reckoner

Leibniz construyó la primera máquina capaz de realizar las cuatro operaciones aritméticas básicas, y publicó "Explication de l'Arithmétique Binaire" (1703), formalizando el sistema binario. Su trabaj...

IMPACTO: REVOLUCIONARIO

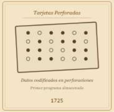
ERA III
Industrial

1700 – 1900

1725

Tarjetas Perforadas para Telares

Basile Bouchon – Lyon, Francia


Tarjetas Perforadas
Datos codificados en perforaciones
Primer programa almacenado
1725

Bouchon inventó un sistema de cintas perforadas para controlar los patrones del telar textil. Este fue el primer uso conocido de un medio de almacenamiento externo para controlar una...

IMPACTO: FUNDAMENTAL

1801

Telar de Jacquard

Joseph Marie Jacquard – Lyon, Francia

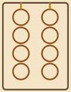
Jacquard perfeccionó el telar automático controlado por tarjetas perforadas, capaz de tejer patrones complejos sin intervención humana. Cada tarjeta representaba una instrucción: subir o baja...

IMPACTO: REVOLUCIONARIO

1822

Máquina Diferencial

Charles Babbage – Londres, Inglaterra



Maquina Diferencial - 1822

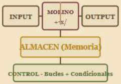
Babbage diseñó la Máquina Diferencial para calcular tablas matemáticas de forma automática, eliminando los errores humanos que plagaban las tablas de navegación y artillería. Aunque nunca s...

`IMPACTO: REVOLUCIONARIO`

1837

Máquina Analítica

Charles Babbage – Londres, Inglaterra


INPUT
MOLINO
+-x/
OUTPUT
ALMACEN (Memoria)
CONTROL - Bucles + Condicionales

Primera computadora completa - 1837

El diseño más ambicioso de Babbage: una máquina de propósito general con unidad aritmético-lógica (ALU), memoria ("almacén"), unidad de control ("molino") y entrada/salida mediante tarjetas...

`IMPACTO: REVOLUCIONARIO`

1843

Primer Algoritmo — Nota G

Ada Lovelace – Londres, Inglaterra

NOTA G

```
V0 = V1 * V2
V4 = V4 - V5
IF V6 = 0
GOTO 2
V7 = V6 + V3
PRINT V7
```

Primer algoritmo - Ada Lovelace 1843

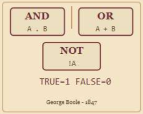
Augusta Ada King, Condesa de Lovelace, tradujo los escritos de Luigi Menabrea sobre la Máquina Analítica y añadió sus propias "Notas". La Nota G contenía un algoritmo para calcular los números ...

`IMPACTO: REVOLUCIONARIO`

1847

Álgebra Booleana

George Boole — Inglaterra


AND
A . B
OR
A + B
NOT
!A
TRUE=1 FALSE=0
George Boole - 1847

Boole publicó "The Mathematical Analysis of Logic", formalizando la lógica mediante expresiones algebraicas con valores de verdadero/falso (1/0). Las operaciones AND, OR,...

`IMPACTO: REVOLUCIONARIO`

1876

Invención del Teléfono

Alexander Graham Bell — Boston, EE.UU.

Bell - 1876
Telecomunicaciones

El teléfono inauguró la era de las telecomunicaciones eléctricas, permitiendo la transmisión instantánea de voz a distancia. Más allá de la comunicación, creó la infraestructura de...

`IMPACTO: ALTO`

1890

Máquina Tabuladora

Herman Hollerith — Washington, EE.UU.

Hollerith = IBM
Tabuladora - Censo 1890

Hollerith creó una máquina electromecánica de tarjetas perforadas para procesar los datos del censo de Estados Unidos de 1890. Redujo el tiempo de procesamiento de 8 años a solo 1 año. Este éxit...

`IMPACTO: TRANSFORMADOR`

ERA IV

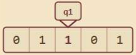
Fundamentos

1900 — 1950

1936

La Máquina de Turing

Alan Turing – Universidad de Cambridge, Inglaterra



Alan Turing ? 1936

On Computable Numbers

En su artículo "On Computable Numbers", Turing describió un dispositivo teórico (la Máquina de Turing) capaz de computar cualquier función computable. Demostró también el problema de la...

`IMPACTO: REVOLUCIONARIO`

1937

Computadora ABC

John V. Atanasoff y Clifford Berry – Iowa, EE.UU.

Atanasoff-Berry 1937

Primera computadora electrónica

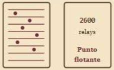
La Atanasoff-Berry Computer (ABC) fue la primera computadora electrónica digital, utilizando tubos de vacío para realizar cálculos. Aunque era de propósito específico (resolver sistemas de...

`IMPACTO: ALTO`

1941

Z3 — Primera Computadora Programable Completa

Konrad Zuse – Berlín, Alemania


2600
relays
Punto
flotante

Konrad Zuse - 1941

Primera computadora programable

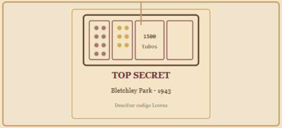
La Z3 fue la primera computadora programable completamente funcional del mundo. Utilizaba relés electromagnéticos, aritmética binaria de punto flotante y era controlada mediante cinta de...

`IMPACTO: REVOLUCIONARIO`

1943

Colossus — Criptoanálisis en Bletchley Park

Tommy Flowers — Bletchley Park, Inglaterra


1500
tubos
TOP SECRET
Bletchley Park - 1943
Descifrar código Lorenz

Colossus fue construida para descifrar los mensajes encriptados de la máquina Lorenz del ejército alemán durante la Segunda Guerra Mundial. Utilizando 1,500 tubos de vacío, fue la primera...

`IMPACTO: CRÍTICO`

1944

Harvard Mark I (ASCC)

Howard Aiken / IBM — Harvard, EE.UU.

16m - 5 toneladas - 765,000 piezas
"Bug" - Grace Hopper
Harvard Mark I - 1944

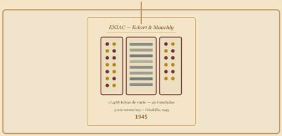
El Mark I fue una computadora electromecánica de 5 toneladas y 16 metros de largo, con 765,000 componentes. Podía realizar 3 sumas por segundo. Grace Hopper, una de sus primeras programadora...

`IMPACTO: ALTO`

1945

ENIAC — Primera Computadora Electrónica de Propósito General

J. Presper Eckert y John Mauchly — Filadelfia, EE.UU.


ENIAC — Eckert & Mauchly
17,468 tubos de vacío — 30 toneladas
5,000 sumas/seg — Filadelfia, 1945
1945

El ENIAC (Electronic Numerical Integrator and Computer) usaba 17,468 tubos de vacío, pesaba 30 toneladas y consumía 150 kW. Podía realizar 5,000 sumas por segundo, mil veces más rápido que...

`IMPACTO: REVOLUCIONARIO`

1945

Arquitectura de Von Neumann

John von Neumann — Princeton, EE.UU.

Arquitectura Von Neumann
CPU
BUS DE DATOS
MEMORIA
INPUT/
OUTPUT
Programa almacenado en memoria
1945

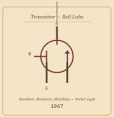
En el informe "First Draft of a Report on the EDVAC", Von Neumann describió una arquitectura donde el programa y los datos comparten la misma memoria. Esta arquitectura — con CPU, memoria...

IMPACTO: REVOLUCIONARIO

1947

Invención del Transistor

John Bardeen, Walter Brattain, William Shockley — Bell Labs, EE.UU.


Transistor — Bell Labs
B
E
Bardeen, Brattain, Shockley — Nobel 1956
1947

El transistor de punto de contacto reemplazó los tubos de vacío: era más pequeño, más rápido, más fiable, generaba menos calor y consumía menos energía. Sin el transistor, cada computadora actu...

IMPACTO: REVOLUCIONARIO

1948

Teoría Matemática de la Comunicación

Claude Shannon — Bell Labs, EE.UU.

H = -Sum p(x) log p(x)

Entropía de Shannon

bit

Unidad fundamental de informacion

Claude Shannon - Bell Labs 1948

Shannon publicó "A Mathematical Theory of Communication", fundando la Teoría de la Información. Introdujo el concepto de "bit" como unidad fundamental de información y demostró...

IMPACTO: REVOLUCIONARIO

ERA V

Consolidación

1950 — 1970

1951

UNIVAC I — Primera Computadora Comercial

Eckert y Mauchly / Remington Rand – EE.UU.

UNIVAC I
UNIVAC
Primera computadora comercial
Predijo victoria de Eisenhower
46 unidades vendidas
1951

El UNIVAC I fue la primera computadora comercial producida en masa. Predijo correctamente la victoria de Eisenhower en las elecciones de 1952, demostrando al público el poder de la...

IMPACTO: TRANSFORMADOR

1956

Primer Disco Duro — IBM RAMAC 305

IBM – San José, California

IBM RAMAC — 50 discos — 1 tonelada — 1956

El IBM 305 RAMAC (Random Access Method of Accounting and Control) introdujo el primer disco duro de la historia: 50 discos de 24 pulgadas que almacenaban 5 MB de datos. Pesaba más de una...

IMPACTO: TRANSFORMADOR

1957

FORTRAN — Primer Lenguaje de Alto Nivel

John Backus y equipo – IBM, EE.UU.

```
PROGRAM HELLO
REAL X, Y, Z
Z = X**2 + Y**2
WRITE(*,*) Z
END
```

Primer lenguaje de alto nivel — IBM 1957

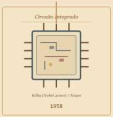
FORTRAN (FORmula TRANslation) fue el primer lenguaje de programación de alto nivel ampliamente utilizado. Antes de FORTRAN, toda programación se hacía en código máquina o...

IMPACTO: REVOLUCIONARIO

1958

Circuito Integrado

Jack Kilby (Texas Instruments) / Robert Noyce (Fairchild)


Circuito Integrado
Kilby (Nobel 2000) / Noyce
1958

De forma independiente, Kilby y Noyce inventaron el circuito integrado: múltiples transistores fabricados en un solo chip de semiconductor. Kilby recibió el Premio Nobel de Física en 2000. El...

IMPACTO: REVOLUCIONARIO

1959

COBOL — Lenguaje para los Negocios

Grace Hopper y Comité CODASYL – EE.UU.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. BANKING.
ADD DEPOSIT TO BALANCE
DISPLAY BALANCE.
```

70% transacciones bancarias

Grace Hopper - 1959

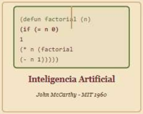
COBOL (COmmon Business-Oriented Language) fue diseñado para ser legible como inglés, facilitando su uso en aplicaciones comerciales. Grace Hopper, la "Madre de COBOL", lideró los...

IMPACTO: TRANSFORMADOR

1960

LISP — Inteligencia Artificial

John McCarthy – MIT, EE.UU.



```
(defun factorial (n)
(if (= n 0)
1
(* n (factorial
(- n 1)))))
```

Inteligencia Artificial

John McCarthy - MIT 1960

LISP (LISt Processing) fue creado para investigación en inteligencia artificial. Introdujo conceptos revolucionarios como la recolección de basura (garbage collection), las funciones como...

IMPACTO: TRANSFORMADOR

1962

Spacewar! — Primer Videojuego

Steve Russell – MIT, EE.UU.

Spacewar! — 1962
Steve Russell — MIT — PDP-1

Spacewar! fue el primer videojuego interactivo significativo, ejecutándose en una PDP-1. Dos jugadores controlaban naves espaciales intentando destruirse mutuamente mientras evitaban la...

IMPACTO: ALTO

1964

IBM System/360 — Familia de Computadoras Compatibles

IBM – Armonk, Nueva York

Model 30
Model 50
Model 75
COMPATIBILIDAD ASCENDENTE
$5 mil millones
IBM 1964

El System/360 fue la primera familia de computadoras con compatibilidad ascendente: el software escrito para un modelo menor funcionaba en uno mayor. IBM invirtió $5 mil millones...

IMPACTO: REVOLUCIONARIO

1965

Ley de Moore

Gordon Moore – Fairchild Semiconductor, EE.UU.

Moore predijo que el número de transistores en un circuito integrado se duplicaría aproximadamente cada dos años, manteniendo el costo constante. Esta "ley" se ha cumplido durante más de 50 años...

IMPACTO: TRANSFORMADOR

1968

Conferencia NATO — Nacimiento de la Ingeniería de Software

Fritz Bauer, Peter Naur y comité — Garmisch, Alemania

SOFTWARE CRISIS
X Exceso de presupuesto
X Exceso de tiempo
X Baja calidad
= Ingeniería de Software
Nace la disciplina - NATO 1968
Garmisch, Alemania

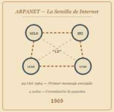
La conferencia NATO en Garmisch, Alemania, acuñó oficialmente el término "Ingeniería de Software" (Software Engineering). Se convocó en respuesta a la "Crisis del Software": los proyectos...

IMPACTO: REVOLUCIONARIO

1969

ARPANET — La Semilla de Internet

DARPA / Bolt, Beranek and Newman — EE.UU.


ARPANET — La Semilla de Internet
UCLA
SRI
"LO"
UCSB
UTAH
29 Oct 1969 — Primer mensaje enviado
4 nodos — Conmutación de paquetes
1969

El 29 de octubre de 1969, se envió el primer mensaje a través de ARPANET desde UCLA a SRI (Stanford Research Institute). El mensaje era "LOGIN" pero la computadora se colgó después d...

IMPACTO: REVOLUCIONARIO

1969

UNIX — El Sistema Operativo Universal

Ken Thompson y Dennis Ritchie — Bell Labs, EE.UU.

```
$ ls -la /
drwxr-xr-x bin
drwxr-xr-x usr
drwxr-xr-x etc
```

"Haz una cosa y hazla bien"
Thompson y Ritchie - Bell Labs
Linux, macOS, iOS, Android - 1969

UNIX fue creado como un sistema operativo multitarea y multiusuario, diseñado con la filosofía de hacer "una cosa y hacerla bien". Su diseño modular, el sistema de archivos jerárquico, los...

IMPACTO: REVOLUCIONARIO

ERA VI
Microprocesador
1970 — 1985

1970

Modelo Relacional de Bases de Datos

Edgar F. Codd – IBM, San José, California

Bases de Datos Relacionales

ID	NOMBRE	DEPT
001	Codd, E.	IBM
002	Date, C.	IBM

```
SELECT * FROM empleados
WHERE dept = 'IBM';
```

Tablas, claves, relaciones
MySQL, PostgreSQL, Oracle, SQL Server
1970

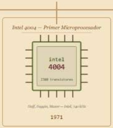
Codd publicó "A Relational Model of Data for Large Shared Data Banks", proponiendo organizar los datos en tablas relacionadas mediante claves. Este modelo es la base de todos los sistemas de gestió...

IMPACTO: REVOLUCIONARIO

1971

Intel 4004 — Primer Microprocesador

Ted Hoff, Federico Faggin, Stanley Mazor – Intel, EE.UU.


Intel 4004 — Primer Microprocesador
intel
4004
2300 transistores
Hoff, Faggin, Mazor — Intel, 740 kHz
1971

El Intel 4004 integró una CPU completa en un solo chip de 10 µm con 2,300 transistores, operando a 740 kHz. Fue diseñado originalmente para una calculadora japonesa (Busicom). El...

IMPACTO: REVOLUCIONARIO

1972

Lenguaje C

Dennis Ritchie – Bell Labs, EE.UU.

C
Dennis Ritchie — Bell Labs
Padre de C++, Java, Python, JavaScript — 1972

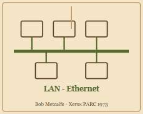
C fue diseñado para reescribir UNIX en un lenguaje de alto nivel portable. Combinó la eficiencia del ensamblador con la legibilidad de un lenguaje de alto nivel. C se convirtió en el lenguaje más...

IMPACTO: REVOLUCIONARIO

1973

Ethernet — Red de Área Local

Bob Metcalfe — Xerox PARC, California


LAN - Ethernet
Bob Metcalfe - Xerox PARC 1973

Metcalfe inventó Ethernet en Xerox PARC, inspirado en ALOHAnet de la Universidad de Hawái. Ethernet permitió conectar computadoras dentro de un edificio a alta velocidad, creando las...

`IMPACTO: TRANSFORMADOR`

1974

TCP/IP — Protocolo de Internet

Vint Cerf y Bob Kahn — DARPA, EE.UU.

Aplicacion (HTTP, FTP)
Transporte (TCP/UDP)
Internet (IP)
Enlace (Ethernet, WiFi)
Vint Cerf y Bob Kahn - 1974

Cerf y Kahn publicaron "A Protocol for Packet Network Intercommunication", definiendo TCP/IP: el conjunto de protocolos que permite la comunicación entre redes diferentes. TCP/IP es...

`IMPACTO: REVOLUCIONARIO`

1975

Altair 8800 y Fundación de Microsoft

Ed Roberts / Bill Gates y Paul Allen — EE.UU.

BASIC
Micro-Soft (1975)
Gates y Allen
Altair 8800 - Kit $439

El Altair 8800, basado en el Intel 8080, fue la primera computadora personal exitosa comercialmente (vendida como kit por $439). Bill Gates y Paul Allen escribieron un intérprete BASIC...

`IMPACTO: TRANSFORMADOR`

1976

Apple I

Steve Wozniak y Steve Jobs — Los Altos, California

$666.66
Wozniak y Jobs — Garaje 1976

Steve Wozniak diseñó el Apple I, una computadora personal que se vendía como placa de circuito por $666.66. Al año siguiente, el Apple II (1977) se convirtió en un éxito masivo con pantalla a color,...

IMPACTO: TRANSFORMADOR

1981

IBM PC — La Computadora Personal Estándar

Don Estridge / IBM — Boca Ratón, Florida

IBM PC — Modelo 5150
1981

El IBM PC (Modelo 5150) utilizó el procesador Intel 8088 y el sistema operativo MS-DOS de Microsoft. Su arquitectura abierta permitió que otros fabricantes crearan "clones", estandarizando la...

IMPACTO: REVOLUCIONARIO

1983

C++ — Programación Orientada a Objetos

Bjarne Stroustrup — Bell Labs, EE.UU.

C++
Clases
Herencia
Polimorfismo
Bjarne Stroustrup — Bell Labs
POO — 1983

Stroustrup extendió C con clases, herencia, polimorfismo y encapsulamiento, creando C++ ("C con Clases"). La programación orientada a objetos (POO) revolucionó el desarrollo de software al...

IMPACTO: TRANSFORMADOR

1984

Macintosh — La Interfaz Gráfica para Todos

Steve Jobs / Apple — Cupertino, California

File Edit View
Doc
Trash
GUI para todos - 1984
Steve Jobs - Apple

El Macintosh popularizó la interfaz gráfica de usuario (GUI) con ventanas, iconos, menús y puntero (paradigma WIMP), inspirada en el Xerox Alto. El famoso anuncio "1984" de Ridley Scott lo...

IMPACTO: TRANSFORMADOR

985

indows 1.0

icrosoft — Redmond, Washington

MS-DOS Executive
Notepad
Calendar
Paint
Windows 1.0 - 1985
Microsoft · 90%+ del mercado

icrosoft lanzó Windows 1.0 como una extensión
áfica de MS-DOS. Aunque inicialmente recibió
íticas, la estrategia de Microsoft de licenciar
indows a múltiples fabricantes de hardware le...

IMPACTO: TRANSFORMADOR

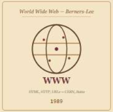
ERA VII
Internet

1989 — 2005

1989

World Wide Web

Tim Berners-Lee — CERN, Suiza


World Wide Web — Berners-Lee
WWW
1989

Berners-Lee propuso un sistema de hipertexto distribuido para compartir información entre científicos del CERN. Inventó HTML, HTTP, URLs y creó el primer navegador web y servidor web. La...

IMPACTO: REVOLUCIONARIO

1991

Linux — El Kernel Libre

Linus Torvalds — Helsinki, Finlandia

Linux — Linus Torvalds
LINUX
1991

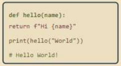
Un estudiante de 21 años publicó el código fuente de un nuevo kernel en un grupo de Usenet: "Estoy haciendo un sistema operativo (libre) [...] solo un hobby". Linux, combinado con las herramientas...

IMPACTO: REVOLUCIONARIO

1991

Python

Guido van Rossum – CWI, Países Bajos



```
def hello(name):
return f"Hi {name}"
print(hello("World"))
# Hello World!
```

#1 en IA y Data Science

Guido van Rossum - 1991

Van Rossum creó Python con la filosofía de que el código debe ser legible y elegante. Su sintaxis limpia y su versatilidad lo convirtieron en uno de los lenguajes más populares del mundo. Python e...

`IMPACTO: TRANSFORMADOR`

1993

Navegador Mosaic

Marc Andreessen y Eric Bina – NCSA, Illinois

NCSA Mosaic
http://www.ncsa.uiuc.edu/
[imagen]
Welcome!
Click here

Imagenes + texto en la web

Andreessen - NCSA 1993

Mosaic fue el primer navegador web con interfaz gráfica que mostraba imágenes junto con el texto en la misma página. Hizo la Web accesible al público general y desencadenó el crecimiento...

`IMPACTO: TRANSFORMADOR`

1995

Java — "Write Once, Run Anywhere"

James Gosling – Sun Microsystems, EE.UU.

"Write Once Run Anywhere"

James Gosling - Sun 1995

Java introdujo la Máquina Virtual Java (JVM), permitiendo ejecutar el mismo código en cualquier plataforma. Su recolección de basura automática, fuerte tipado y enfoque orientado a objetos lo...

`IMPACTO: REVOLUCIONARIO`

1995

JavaScript — El Lenguaje de la Web

Brendan Eich — Netscape, EE.UU.

JS
Creado en 10 días
#1 lenguaje mas usado
Brendan Eich - Netscape 1995
React, Angular, Vue, Node.js

Brendan Eich creó JavaScript en solo 10 días para Netscape Navigator. A pesar de sus orígenes apresurados, se convirtió en el único lenguaje de programación que ejecutan nativamente todos lo...

IMPACTO: REVOLUCIONARIO

1998

Google — Organizar la Información del Mundo

Larry Page y Sergey Brin — Stanford, California

Google
buscar en la web...
PageRank
Page y Brin - Stanford 1998
8,5 mil millones busquedas/dia

Page y Brin desarrollaron PageRank, un algoritmo que clasificaba las páginas web por la cantidad y calidad de enlaces que recibían, dentro de su proyecto doctoral en Stanford. Google transformó...

IMPACTO: REVOLUCIONARIO

2001

Manifiesto Ágil

17 desarrolladores — Snowbird, Utah, EE.UU.

Manifesto for Agile Software Development
Individuos
sobre procesos
Software funcional
sobre docs
Colaboracion
sobre contrato
Respuesta al cambio
sobre pla
Snowbird Utah - 17 firmantes - 2001

Diecisiete expertos en desarrollo de software firmaron el Manifiesto Ágil, estableciendo 4 valores y 12 principios que priorizan individuos sobre procesos, software funcional sobre documentació...

IMPACTO: TRANSFORMADOR

2004

Facebook y la Web 2.0

Mark Zuckerberg — Harvard, Massachusetts

f
Web 2.0 - React GraphQL
Zuckerberg - Harvard 2004

Facebook representó la culminación de la "Web 2.0": plataformas donde los usuarios generan el contenido. Demostró que las aplicaciones web podían escalar a miles de millones de usuarios y...

`IMPACTO: TRANSFORMADOR`

ERA VIII
Móvil y Nube
2007 — 2019

2007

iPhone — La Revolución Móvil

Steve Jobs — Apple, Cupertino, California

iPhone — Steve Jobs
App Store — [illegible] anuales
2007

Jobs presentó el iPhone como "un iPod, un teléfono y un dispositivo de comunicación de Internet" en uno. Su pantalla multitáctil y la App Store (2008) crearon un nuevo paradigma de computación y u...

`IMPACTO: REVOLUCIONARIO`

2008

Android y GitHub

Google / Chris Wanstrath, Tom Preston-Werner — EE.UU.

Android 72%
GitHub 100M+
Open Source + Colaboracion
2008

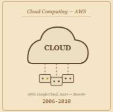
Android (basado en Linux) democratizó los smartphones al ser un sistema operativo abierto disponible para cualquier fabricante, alcanzando el 72% del mercado global. GitHub revolucionó el...

`IMPACTO: REVOLUCIONARIO`

2006-2010

Computación en la Nube

Amazon (AWS) / Google / Microsoft – EE.UU.


Cloud Computing – AWS
CLOUD
2006-2010

Amazon Web Services (AWS) lanzó EC2 en 2006, inaugurando la era de la computación en la nube. En lugar de comprar servidores, las empresas podían alquilar capacidad de cómputo por minut...

IMPACTO: REVOLUCIONARIO

2011

IBM Watson — IA Vence en Jeopardy!

IBM Research – Yorktown Heights, Nueva York

JEOPARDY!
Ken
$21,600
WATSON
$77,147
Brad
$24,000
IA vencio a humanos
IBM Watson - 2011

IBM Watson venció a los dos mejores concursantes humanos en el programa de televisión Jeopardy!, demostrando que la IA podía procesar lenguaje natural y razonar sobre conocimiento general....

IMPACTO: ALTO

2013

Docker — Contenedores y Microservicios

Solomon Hykes – dotCloud, Francia/EE.UU.

Microservicios
Solomon Hykes - DevOps CI/CD - 2013

Docker popularizó los contenedores Linux, permitiendo empaquetar aplicaciones con todas sus dependencias en unidades portátiles y reproducibles. Combinado con Kubernetes...

IMPACTO: TRANSFORMADOR

2014

Kubernetes — Orquestación de Contenedores

Google (Brendan Burns, Joe Beda, Craig McLuckie) – EE.UU.

K8s
Orquestacion de contenedores
Google - 96% de organizaciones - 2014

Google liberó Kubernetes como proyecto de código abierto, basado en más de 15 años de experiencia operando contenedores internamente (Borg). Kubernetes automatiza el despliegue, escalado y...

IMPACTO: TRANSFORMADOR

ERA IX
Inteligencia Artificial

2016 – Presente

2016

AlphaGo Vence al Campeón Mundial de Go

DeepMind (Google) – Londres, Reino Unido

AlphaGo – DeepMind
A.I.
Deep Learning venció al campeón mundial de Go
2016

AlphaGo venció 4-1 a Lee Sedol, campeón mundial de Go, un juego considerado demasiado complejo para que las computadoras lo dominaran (más posiciones que átomos en el universo). Utilizando...

IMPACTO: REVOLUCIONARIO

2017

Transformers — "Attention Is All You Need"

Vaswani et al. – Google Brain, EE.UU.

Self-Attention
Q — K — V
Encoder
Decoder
"Attention Is All You Need"
Google Brain - Base de GPT, BERT, LLaMA
Vaswani et al. - 2017

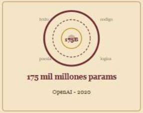
El artículo "Attention Is All You Need" introdujo la arquitectura Transformer, que reemplazó las redes neuronales recurrentes para el procesamiento de lenguaje natural. El mecanismo de "atención"...

IMPACTO: REVOLUCIONARIO

2020

GPT-3 — Modelos de Lenguaje a Gran Escala

OpenAI — San Francisco, California


texto
codigo
175B
poesia
logica
175 mil millones params
OpenAI - 2020

GPT-3, con 175 mil millones de parámetros, demostró capacidades emergentes sorprendentes: podía escribir ensayos, código, poesía, traducir idiomas y razonar sobre problemas. Se entrenó co...

IMPACTO: TRANSFORMADOR

2021

GitHub Copilot — IA como Coprogramador

GitHub / OpenAI — EE.UU.

```
def fibonacci(n):
# Copilot sugiere:
if n <= 1:
[illegible]
return fib(n-1)
+ fib(n-2)
```

55% mas rapido con IA
GitHub + OpenAI - 2021

GitHub Copilot, basado en el modelo Codex (derivado de GPT-3), se convirtió en el primer asistente de programación con IA ampliamente adoptado. Sugiere código completo, funciones y...

IMPACTO: TRANSFORMADOR

Noviembre 2022

ChatGPT — IA Accesible para Todos

OpenAI — San Francisco, California

Que es la ingenieria de software?

La ingenieria de software es una disciplina que aplica principios de ingenieria al desarrollo de software

100M
usuarios en 2 meses
OpenAI - Nov. 2022

ChatGPT, basado en GPT-3.5, alcanzó 100 millones de usuarios en solo 2 meses, el crecimiento más rápido en la historia de las aplicaciones de consumo. Demostró que la IA conversacional pod...

IMPACTO: REVOLUCIONARIO

Marzo 2023

GPT-4 — IA Multimodal

OpenAI — San Francisco, California

Texto
in/out
Imagen
input
Supera humanos en:
Examen abogacía (P90), SAT, GRE
IA Multimodal
OpenAI - Marzo 2023

GPT-4 introdujo capacidades multimodales (texto e imagen), razonamiento avanzado y puntuó en el percentil 90 del examen de abogacía de EE.UU. Supera a humanos en múltiples exámenes...

`IMPACTO: REVOLUCIONARIO`

2024 — 2026

IA Generativa Masiva y Agentes Autónomos

OpenAI, Google, Anthropic, Meta, Microsoft — Global

Texto
Video
AGI?
Codigo
Musica
Agentes autonomos de IA
Regulacion - Etica - Empleo
2024-2026

La IA generativa se integra en prácticamente todas las herramientas de software: IDEs, diseño gráfico, música, video, análisis de datos. Los agentes de IA autónomos pueden navegar la web, ejecutar...

`IMPACTO: CRÍTICO`

2.4 Análisis Contrafáctico: ¿Qué habría ocurrido sin el Transistor y ARPANET?

El análisis contrafáctico la especulación razonada sobre qué habría sucedido si ciertos eventos históricos no hubieran ocurrido es una herramienta metodológica valiosa para comprender la importancia causal de los hitos tecnológicos. Se presentan a continuación dos escenarios contrafácticos.

Escenario A: Sin el Transistor

Si el transistor no hubiera sido inventado en el siglo XX o si su desarrollo se hubiera retrasado varias décadas, la computación habría permanecido en la era de los tubos de vacío. Las consecuencias para la Ingeniería de Software habrían sido múltiples. La computación habría continuado siendo una actividad exclusivamente institucional: costosa, frágil y de difícil acceso. Sin la miniaturización que el transistor hizo posible, no habría microprocesadores, ni computadoras personales, ni dispositivos móviles. El mercado de software habría permanecido siendo un mercado de nicho, y la presión para desarrollar metodologías formales de gestión de proyectos de software habría sido considerablemente menor. La crisis del software de los años 1960-1970 podría no haber tenido la escala necesaria para justificar la creación de la Ingeniería de Software como disciplina formal.

Adicionalmente, sin la miniaturización del transistor, la Ley de Moore no habría tenido sustento físico. El crecimiento exponencial de la capacidad de cómputo que esta ley describe y que ha sido el principal motor de la innovación en software durante más de cinco décadas no habría ocurrido. La historia de la Ingeniería de Software sería, en este escenario, la historia de una disciplina que gestionaba sistemas de complejidad moderada y estable, en lugar de sistemas cuya complejidad se duplicaba aproximadamente cada dos años.

Escenario B: Sin ARPANET

Si ARPANET no hubiera sido creada, o si el desarrollo de Internet se hubiera retrasado significativamente, la Ingeniería de Software habría evolucionado por un camino radicalmente diferente. Sin la red, el software habría permanecido siendo un fenómeno local durante décadas más. Esto habría implicado, en primer lugar, la ausencia de los desafíos de distribución que generaron algunos de los avances más significativos en la disciplina: los sistemas de bases de datos distribuidas, la arquitectura de microservicios, los protocolos de consistencia eventual y los sistemas tolerantes a fallos.

En segundo lugar, sin Internet, el movimiento de software libre y de código abierto no habría alcanzado la escala que tiene. Linux, que ejecuta el 96% de los servidores web del mundo, dependió de Internet para su desarrollo colaborativo distribuido. GitHub, que alberga el código de más de 100 millones de desarrolladores, es inconcebible sin red. La imposibilidad de compartir código a escala global habría ralentizado enormemente la acumulación del conocimiento colectivo que hoy constituye el ecosistema de software libre.

En tercer lugar, sin la presión de escala que Internet introdujo, metodologías como DevOps, la integración continua, el despliegue continuo y la orquestación de contenedores (Docker, Kubernetes) probablemente no habrían surgido. Estas metodologías nacieron de la necesidad de operar sistemas de software a escala planetaria con equipos distribuidos globalmente, una necesidad que no existiría en un mundo sin Internet.

El análisis contrafáctico confirma, por lo tanto, que el transistor y ARPANET no son solo los hitos tecnológicos más importantes de la historia de la computación: son las

condiciones de posibilidad sin las cuales la Ingeniería de Software, tal como la define Pressman y Maxim (2020) y tal como la practica la industria del software del siglo XXI, simplemente no existiría.

III. ACTIVIDADES COMPLEMENTARIAS

3.1 Ensayo: El Alan Turing del Futuro

En 1936, un joven matemático de 24 años de la Universidad de Cambridge publicó un artículo titulado "On Computable Numbers, with an Application to the Entscheidungsproblem". El artículo era, en apariencia, una respuesta a un problema técnico de lógica matemática planteado por David Hilbert. En la práctica, Alan Turing estaba describiendo la arquitectura conceptual de toda computadora que existiría en los siguientes cien años. Este es el patrón que define al genio transformador en la historia de la tecnología: la solución de un problema abstracto con consecuencias prácticas de alcance civilizacional.

¿Quién será el Alan Turing del futuro? La pregunta no es meramente especulativa, sino metodológicamente necesaria para cualquier ingeniero de software que pretenda no solo adaptarse a la historia tecnológica, sino participar activamente en su construcción. La respuesta requiere primero comprender qué hizo de Turing una figura singular, más allá de su indudable genialidad matemática.

Turing fue, en esencia, un constructor de puentes entre la abstracción y la implementación. La Máquina de Turing no era una propuesta de ingeniería: era un instrumento de análisis matemático. Sin embargo, proporcionó el modelo formal que legitimó el desarrollo de computadoras físicas. Von Neumann leyó el trabajo de Turing antes de formular su arquitectura. Church, Kleene y Post construyeron sobre sus fundamentos. La computación teórica y la computación práctica son, desde Turing, una sola disciplina.

El Alan Turing del futuro deberá construir un puente similar, pero entre dominios más complejos. Los grandes problemas no resueltos de la computación contemporánea la inteligencia artificial general, la

computación cuántica escalable, la biología computacional, la gestión de sistemas de software de escala planetaria comparten la característica de que su solución requerirá avances teóricos que preceded a los avances prácticos, exactamente como ocurrió con Turing. El Turing del futuro puede ser la investigadora que formalice matemáticamente qué significa que un sistema de inteligencia artificial "comprende" en lugar de solo correlacionar, de la misma manera que Turing formalizó qué significa que una máquina "computa".

Puede también ser el ingeniero que resuelva el problema de la verificación formal de sistemas de software complejos: demostrar matemáticamente que un programa no contiene errores, del mismo modo que un matemático demuestra un teorema. Este problema, conocido como el problema de la corrección del programa, es en cierto sentido el heredero directo del problema de la detención que Turing demostró irresoluble en su forma general. La solución parcial o aproximada de este problema transformaría la Ingeniería de Software de la misma manera en que la arquitectura de Von Neumann transformó la computación.

Lo que resulta claro, desde la perspectiva de la historia de la tecnología analizada en esta práctica, es que el Alan Turing del futuro no será simplemente el creador de la herramienta más poderosa del momento, sino aquel que proporcione el marco conceptual que haga posibles las herramientas del siguiente siglo. El genio transformador no es el que construye sobre los fundamentos existentes: es el que construye los fundamentos mismos. Para el estudiante de Ingeniería de Software, esta observación tiene una implicación directa: la formación matemática, teórica y humanística no es un lujo académico, sino la condición necesaria para participar en la historia, y no solo en el mercado laboral.

3.2 Reflexión: Ada Lovelace y su Visión de la Inteligencia Artificial

En 1843, Augusta Ada King Condesa de Lovelace, hija del poeta Lord Byron y protegida intelectual de Charles Babbage publicó sus Notas sobre la traducción del artículo de Luigi Menabrea acerca de la

Máquina Analítica. Las Notas, especialmente la Nota G, son reconocidas universalmente como el primer programa de computadora de la historia: un algoritmo para calcular los números de Bernoulli. Pero lo más sorprendente del texto de Lovelace no es su contribución algorítmica, sino sus reflexiones filosóficas sobre los límites y posibilidades de las máquinas calculadoras.

En un párrafo que ha sido citado y debatido durante más de 150 años, Lovelace escribió que la Máquina Analítica "no tiene pretensiones de originar nada. Puede hacer lo que sea que sepamos ordenarle que realice. Puede analizar, pero no tiene poder para originar conclusiones analíticas. Ese poder pertenece exclusivamente a lo mental". Esta afirmación, conocida en la filosofía de la mente como la "Objeción de Lady Lovelace", fue retomada y rebatida por el propio Turing en su artículo seminal de 1950, "Computing Machinery and Intelligence", donde propuso el Test de Turing como criterio operacional de inteligencia.

Lo fascinante de Lovelace no es solo que describiera los límites de las máquinas en 1843, sino que simultáneamente identificara su potencial transformador. Fue la primera en reconocer que la Máquina Analítica podía manipular cualquier símbolo y no solo números: "La Máquina Analítica podría actuar sobre otras cosas que no fueran el número, si fueran encontradas cuya relaciones fundamentales mutuas pudieran ser expresadas por las de la ciencia abstracta de las operaciones". Este insight que la computación es fundamentalmente sobre la manipulación simbólica y no sobre el cálculo numérico anticipa en más de un siglo la teoría de la información de Shannon, los lenguajes de programación simbólica como LISP, y la inteligencia artificial como procesamiento de lenguaje natural.

Desde la perspectiva de 2025, la visión de Lovelace adquiere una dimensión extraordinaria. Los sistemas de inteligencia artificial generativa como GPT-4 y sus sucesores realizan exactamente lo que Lovelace describió como teóricamente posible: operan sobre símbolos lingüísticos según reglas abstractas, produciendo texto que en ocasiones es indistinguible del producido por un ser humano. ¿Significa esto que la "Objeción de Lady Lovelace" ha sido refutada? ¿O significa que los LLMs son

precisamente máquinas que hacen lo que se les ordena, pero de manera tan sofisticada que la distinción entre "originar" y "ejecutar" se vuelve filosóficamente difusa?

Esta reflexión tiene implicaciones directas para el ingeniero de software contemporáneo. La visión de Lovelace nos recuerda que las máquinas son instrumentos cuya capacidad está acotada por la calidad de las instrucciones que reciben: los datos de entrenamiento, los algoritmos de optimización, las funciones de pérdida. El ingeniero de software que desarrolla, evalúa o integra sistemas de IA no es fundamentalmente diferente de Ada Lovelace: su trabajo consiste en transformar una intención abstracta resolver un problema del mundo real en una secuencia de instrucciones que una máquina pueda ejecutar. La diferencia es de escala y herramientas, no de naturaleza.

Ada Lovelace también nos enseña algo sobre la visibilidad de las contribuciones en la historia de la tecnología. Durante décadas, su trabajo fue minimizado o ignorado; fue principalmente gracias a B.V. Bowden, quien publicó sus Notas en 1953, que su contribución fue reconocida. El lenguaje de programación Ada, creado por el Departamento de Defensa de los EE.UU. en 1983, fue nombrado en su honor. Su historia es un recordatorio de que la historia de la tecnología, como toda historia, refleja las estructuras de poder de su época, y que la ingeniería del futuro será más robusta en la medida en que incorpore la diversidad de perspectivas que históricamente ha excluido.

3.3 Algoritmo de Fibonacci Representado en Tarjetas Perforadas

El siguiente ejercicio conceptual implementa el algoritmo de Fibonacci utilizando la metáfora de las tarjetas perforadas, el primer medio de almacenamiento de programas de la historia (Bouchon, 1725; Jacquard, 1801). Cada tarjeta representa una instrucción de 8 bits, donde el estado de cada posición (perforado = 1, sin perforar = 0) codifica la operación y sus operandos. Este ejercicio ilustra el principio fundamental de que toda computación es, en última instancia, la manipulación de símbolos binarPROGRAMA: Fibonacci mediante tarjetas perforadas (sistema binario)

TARJETA 01: [0 0 0 0 0 0 0 1] → INICIO. Cargar valor 1 en Registro A

TARJETA 02: [0 0 0 0 0 0 1 0] → Cargar valor 0 en Registro B

TARJETA 03: [0 0 0 0 0 0 1 1] → Imprimir valor de Registro B

TARJETA 04: [0 0 0 0 0 1 0 0] → Copiar Registro B → Registro TEMP

TARJETA 05: [0 0 0 0 0 1 0 1] → SUMA: A + B → Registro B

TARJETA 06: [0 0 0 0 0 1 1 0] → Copiar Registro TEMP → Registro A

TARJETA 07: [0 0 0 0 0 1 1 1] → ¿B > 1000? SI: ir a TARJETA 09

TARJETA 08: [0 0 0 0 1 0 0 0] → SALTO: ir a TARJETA 03 (bucle)

TARJETA 09: [0 0 0 0 1 0 0 1] → FIN. Detener ejecución

TRAZA DE EJECUCIÓN:

Iter 1: A=1, B=0 → Imprime: 0 (00000000)

Iter 2: A=0, B=1 → Imprime: 1 (00000001)

Iter 3: A=1, B=1 → Imprime: 1 (00000001)

Iter 4: A=1, B=2 → Imprime: 2 (00000010)

Iter 5: A=2, B=3 → Imprime: 3 (00000011)

Iter 6: A=3, B=5 → Imprime: 5 (00000101)

Iter 7: A=5, B=8 → Imprime: 8 (00001000)

Iter 8: A=8, B=13 → Imprime: 13 (00001101)

...continúa hasta B > 1000

NOTAS DE DISEÑO:

- *Cada tarjeta = 1 instrucción = 1 byte de 8 bits*
- *Bit 1 (izquierda) = tipo de instrucción: 0=aritmética, 1=control*
- *Bits 2-4 = código de operación (000=carga, 001=suma, 010=copia)*
- *Bits 5-8 = operando (registro destino/fuente o dirección de salto)*
- *El mecanismo de bucle (TARJETA 08) replica el ciclo del telar*

de Jacquard: regresar a la tarjeta de inicio de iteración.

ios según reglas definida

La analogía con el telar de Jacquard no es superficial. En ambos casos, la "programación" consiste en preparar un conjunto ordenado de instrucciones físicas (tarjetas) que el mecanismo ejecutor (telar o computadora) procesa secuencialmente. El concepto de bucle —representado en las tarjetas 07 y 08 del algoritmo anterior— replica directamente el mecanismo de retorno de tarjetas del telar de Jacquard, que permitía repetir un patrón hasta que se cumpliera una condición de terminación. Esta continuidad conceptual entre el telar del siglo XIX y el procesador del siglo XXI es uno de los argumentos más convincentes a favor del estudio histórico de la tecnología para la formación del ingeniero de software.

CONCLUSIONES

El análisis de la evolución tecnológica realizado en esta práctica conduce a una conclusión que trasciende el inventario histórico: la Ingeniería de Software es, en su esencia más profunda, una respuesta a la complejidad. Cada avance tecnológico analizado —desde las primeras herramientas de piedra hasta la arquitectura Transformer— puede interpretarse como la resolución de una forma específica de complejidad, seguida por la introducción de una forma de complejidad de orden superior que requería, a su vez, nuevas soluciones.

La continuidad entre la antigüedad y la informática moderna es más profunda de lo que sugiere una lectura superficial de la historia. El algoritmo de Euclides para calcular el máximo común divisor (aprox. 300 a.C.) es, en todos los aspectos esenciales, lo mismo que un método de una clase Java o una función de Python: una secuencia finita de pasos bien definidos que transforma una entrada en una salida. El sistema decimal posicional con el cero, desarrollado en la India alrededor del siglo III d.C., es el precursor intelectual directo del sistema binario que implementan todos los procesadores modernos. La Máquina Analítica de Babbage (1837) anticipó con asombrosa precisión la arquitectura

de Von Neumann (1945): ambas tienen una unidad aritmético-lógica, memoria, unidad de control y mecanismos de entrada/salida. La historia de la tecnología es, en este sentido, una historia de redescubrimiento: cada generación reinventa, en un contexto material diferente, las mismas ideas fundamentales.

Para el ingeniero de software contemporáneo, esta perspectiva histórica tiene una implicación práctica fundamental: comprender los principios es más valioso que dominar las herramientas específicas. Las herramientas cambian a una velocidad que hace obsoleto cualquier conocimiento técnico concreto en el plazo de pocos años. Los principios —abstracción, modularidad, separación de responsabilidades, gestión de la complejidad, verificación y prueba— son los mismos que articularon Babbage, Lovelace, Turing, Von Neumann, Shannon, Dijkstra y Brooks. La formación del ingeniero que perdura es aquella que enseña a pensar sobre la complejidad, no solo a operar las herramientas que la gestionan en un momento dado.

La Ley de Moore y su ralentización en la última década nos recuerda que el ritmo exponencial de la innovación tecnológica no es una ley de la naturaleza, sino el resultado de condiciones específicas que pueden cambiar. La Ingeniería de Software deberá adaptarse a un mundo en el que el incremento de rendimiento no provenga automáticamente del hardware más rápido, sino de algoritmos más eficientes, arquitecturas de software más inteligentes y el aprovechamiento de paradigmas de cómputo alternativos, como la computación cuántica o la computación neuromórfica. Esta adaptación requerirá exactamente el tipo de pensamiento histórico y teórico que esta práctica ha cultivado.

Finalmente, el surgimiento de la inteligencia artificial generativa como herramienta del ingeniero de software —representada en la línea del tiempo por hitos como GitHub Copilot (2021) y ChatGPT (2022)— introduce una transformación sin precedentes en la naturaleza misma del trabajo del ingeniero. La Ingeniería de Software está migrando, progresivamente, desde la escritura de código hacia la orquestación de sistemas de IA: la especificación precisa de problemas, la evaluación crítica de soluciones generadas automáticamente y la garantía de que los sistemas resultantes sean correctos,

seguros y éticamente responsables. Esta evolución no reduce la importancia del ingeniero: la amplifica, al transferir la carga de la implementación rutinaria a las máquinas y concentrar la responsabilidad humana en las dimensiones que las máquinas aún no pueden gestionar: el juicio, la ética y la comprensión del contexto humano en el que el software opera.

En suma, esta práctica confirma que el estudio de la historia de la tecnología no es una actividad de erudición periférica, sino una competencia central del ingeniero de software moderno. Conocer de dónde venimos es la condición necesaria para entender dónde estamos y, lo más importante, para participar conscientemente en la construcción de donde iremos.

REFERENCIAS

Brooks, F. P. (1995). The mythical man-month: Essays on software engineering (Anniversary ed.). Addison-Wesley Professional.

Ceruzzi, P. E. (2003). A history of modern computing (2nd ed.). MIT Press.

Isaacson, W. (2014). The innovators: How a group of hackers, geniuses, and geeks created the digital revolution. Simon & Schuster.

Knuth, D. E. (1997). The art of computer programming: Vol. 1. Fundamental algorithms (3rd ed.). Addison-Wesley.

Naughton, J. (2000). A brief history of the future: From radio days to internet years in a lifetime. Overlook Press.

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

Sommerville, I. (2011). Software engineering (9th ed.). Addison-Wesley.

Tanenbaum, A. S. (2013). Modern operating systems (4th ed.). Pearson.